

Skill Week- 5

Task 1

To Change Directories, Validate Paths, Update Working Directory, Handle Errors, Maintain Previous Directory, Test Navigation

```

GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <limits.h>
#include <sys/stat.h>

int main() {
    char path[PATH_MAX];
    char current_dir[PATH_MAX];
    char previous_dir[PATH_MAX] = "";
    struct stat st;

    printf("Simple Directory Navigation Program\n");
    printf("Commands: cd <path>, pwd, previous, exit\n\n");

    while (1) {
        if (getcwd(current_dir, sizeof(current_dir)) == NULL) {
            perror("getcwd");
            break;
        }
        printf("%s$ ", current_dir);

        if (fgets(path, sizeof(path), stdin) == NULL) {
            break;
        }
        path[strcspn(path, "\n")] = '\0';

        if (strcmp(path, "exit") == 0) {
            printf("Exiting...\n");
            break;
        }
        if (strcmp(path, "pwd") == 0) {
            printf("Current Directory: %s\n", current_dir);
            continue;
        }
        if (strcmp(path, "previous") == 0) {
            if (strlen(previous_dir) == 0) {
                printf("No previous directory available.\n");
                continue;
            }
            char temp[PATH_MAX];
            strcpy(temp, current_dir);
            if (chdir(previous_dir) == -1) {
                perror("chdir");
                continue;
            }
            strcpy(previous_dir, temp);
            printf("Changed to previous directory.\n");
            continue;
        }
        if (strncmp(path, "cd ", 3) == 0) {
            char *new_path = path + 3;
            while (*new_path == ' ') {
                new_path++;
            }
            if (stat(new_path, &st) == -1) {
                perror("Invalid path");
                continue;
            }
            if (!S_ISDIR(st.st_mode)) {
                printf("Error: Path is not a directory.\n");
                continue;
            }
            strcpy(previous_dir, current_dir);
            if (chdir(new_path) == -1) {
                perror("chdir");
                continue;
            }
            if (getcwd(current_dir, sizeof(current_dir)) == NULL) {
                perror("getcwd");
                continue;
            }
            printf("Directory changed successfully.\n");
            printf("New Working Directory: %s\n", current_dir);
        } else {
            printf("Invalid command.\n");
            printf("Use: cd <path>, pwd, previous, or exit\n");
        }
    }
    return 0;
}

```

OUTPUT:



imvarls@LAPTOP-VQ4D70L7: ×

+

✓

```
imvarls@LAPTOP-VQ4D70L7:~$ nano navigation.c
imvarls@LAPTOP-VQ4D70L7:~$ gcc navigation.c -o navigation
imvarls@LAPTOP-VQ4D70L7:~$ ./navigation
Simple Directory Navigation Program
Commands: cd <path>, pwd, previous, exit

/home/imvarls$ pwd
Current Directory: /home/imvarls
/home/imvarls$ |
```

Task 2

To Identify Built-ins, Create Dispatch Table, Execute In Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_INPUT 100
#define MAX_ARGS 20

void command_pwd(char **args);
void command_cd(char **args);
void command_help(char **args);
void command_exit(char **args);

typedef struct {
    char *name;
    void (*function) (char **args);
} Command;

Command dispatch_table[] = {
    {"pwd", command_pwd},
    {"cd", command_cd},
    {"help", command_help},
    {"exit", command_exit}
};

int command_count = sizeof(dispatch_table) / sizeof(dispatch_table[0]);

void command_pwd(char **args) {
    char cwd[1024];
    if (getcwd(cwd, sizeof(cwd)) != NULL) {
        printf("%s\n", cwd);
    }
}

void command_cd(char **args) {
    if (args[1] == NULL) {
        printf("cd: missing argument\n");
        return;
    }
    if (chdir(args[1]) != 0) {
        perror("cd");
    }
}

void command_help(char **args) {
    printf("\nAvailable commands:\n");
    printf(" pwd      Show current directory\n");
    printf(" cd <dir> Change directory\n");
    printf(" help     Show available commands\n");
    printf(" exit     Exit shell\n");
    printf(" Other commands are executed as processes.\n\n");
}
```

```

GNU nano 8.7.1 shell.c
int execute_builtin(char **args) {
    for (int i = 0; i < command_count; i++) {
        if (strcmp(args[0], dispatch_table[i].name) == 0) {
            dispatch_table[i].function(args);
            return 1;
        }
    }
    return 0;
}

int parse_input(char *input, char **args) {
    int count = 0;
    char *token = strtok(input, " \t\n");
    while (token != NULL && count < MAX_ARGS - 1) {
        args[count++] = token;
        token = strtok(NULL, " \t\n");
    }
    args[count] = NULL;
    return count;
}

int main() {
    char input[MAX_INPUT];
    char *args[MAX_ARGS];

    printf("=====\n");
    printf(" SIMPLE COMMAND SHELL\n");
    printf("=====\n");

    while (1) {
        fflush(stdout);
        printf("myshell> ");

        if (fgets(input, sizeof(input), stdin) == NULL) {
            break;
        }
        if (strlen(input) <= 1) {
            continue;
        }

        int argc = parse_input(input, args);
        if (argc == 0) {
            continue;
        }
        if (execute_builtin(args)) {
            continue;
        }
        execute_process(args);
    }
    return 0;
}

```

[illegible]

To Retrieve Current Directory, Display Path, Process Exit Requests, Cleanup Resources, Save State, Verify Correct Behavior

```

GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <limits.h>
#include <string.h>

void cleanup() {
    printf("\nCleaning up resources...\n");
    printf("Resources cleaned successfully.\n");
}

int main() {
    char cwd[PATH_MAX];
    atexit(cleanup);

    printf("=== Directory and Process Management ===\n\n");

    if (getcwd(cwd, sizeof(cwd)) != NULL) {
        printf("Current Directory: %s\n", cwd);
    } else {
        perror("getcwd");
        return 1;
    }

    printf("Path: %s\n", cwd);

    FILE *file = fopen("state.txt", "w");
    if (file != NULL) {
        fprintf(file, "Saved Working Directory: %s\n", cwd);
        fclose(file);
        printf("State saved successfully.\n");
    } else {
        perror("State file");
    }

    if (getcwd(cwd, sizeof(cwd)) != NULL) {
        printf("Verification: Current directory is correct.\n");
    } else {
        printf("Verification failed.\n");
    }

    char choice;
    printf("\nDo you want to exit? (y/n): ");
    scanf(" %c", &choice);

    if (choice == 'y' || choice == 'Y') {
        printf("Process exit requested.\n");
        exit(0);
    } else {
        printf("Process will continue running.\n");
        return 0;
    }
}

```

OUTPUT:

```
imvarls@LAPTOP-VQ4D70L7:~$ nano directory.c
imvarls@LAPTOP-VQ4D70L7:~$ gcc directory.c -o directory
imvarls@LAPTOP-VQ4D70L7:~$ ./directory
=== Directory and Process Management ===

Current Directory: /home/imvarls
Path: /home/imvarls
State saved successfully.
Verification: Current directory is correct.

Do you want to exit? (y/n): y
Process exit requested.

Cleaning up resources...
Resources cleaned successfully.
imvarls@LAPTOP-VQ4D70L7:~$ |
```

Task 4

To Parse Export Syntax, Update Environment Variables, Validate Names, Handle Existing Variables, Export to Child Processes, Test Functionality

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_INPUT 200
#define MAX_NAME 100
#define MAX_VALUE 100

int valid_name(char *name) {
    if (name == NULL || name[0] == '\0')
        return 0;
    if (!isalpha(name[0]) && name[0] != '_')
        return 0;
    for (int i = 1; name[i] != '\0'; i++) {
        if (!isalnum(name[i]) && name[i] != '_')
            return 0;
    }
    return 1;
}

void export_variable(char *input) {
    char name[MAX_NAME];
    char value[MAX_VALUE];
    char *assignment = input + 7;
    char *equal = strchr(assignment, '=');

    if (equal == NULL) {
        printf("Error: Invalid export syntax.\n");
        printf("Usage: export NAME=VALUE\n");
        return;
    }

    int name_length = equal - assignment;
    if (name_length <= 0 || name_length >= MAX_NAME) {
        printf("Error: Invalid variable name.\n");
        return;
    }

    strncpy(name, assignment, name_length);
    name[name_length] = '\0';
    strcpy(value, equal + 1);

    if (!valid_name(name)) {
        printf("Error: Invalid variable name: %s\n", name);
        return;
    }

    char *old_value = getenv(name);
    if (old_value != NULL) {
        printf("Existing variable found: %s=%s\n", name, old_value);
    }

    if (setenv(name, value, 1) != 0) {
        perror("setenv");
        return;
    }
}
```

GNU nano 8.7.1

```
        return;
    }
    printf("Export successful: %s=%s\n", name, value);
}

void test_child_process() {
    pid_t pid = fork();
    if (pid < 0) {
        perror("fork");
        return;
    }
    if (pid == 0) {
        printf("\nChild Process: \n");
        char *value = getenv("MYVAR");
        if (value != NULL)
            printf("MYVAR=%s\n", value);
        else
            printf("MYVAR is not set.\n");
        exit(0);
    } else {
        wait(NULL);
        printf("Child process completed.\n");
    }
}

int main() {
    char input[MAX_INPUT];
    printf("=====\n");
    printf(" EXPORT COMMAND TESTER\n");
    printf("=====\n");
    while (1) {
        printf("\nshell> ");
        if (fgets(input, sizeof(input), stdin) == NULL)
            break;
        input[strcspn(input, "\n")] = '\0';
        if (strcmp(input, "exit") == 0) {
            printf("Exiting...\n");
            break;
        }
        if (strncmp(input, "export ", 7) == 0) {
            export_variable(input);
        } else if (strcmp(input, "test") == 0) {
            test_child_process();
        } else if (strncmp(input, "print ", 6) == 0) {
            char *name = input + 6;
            char *value = getenv(name);
            if (value != NULL)
                printf("%s=%s\n", name, value);
            else
                printf("%s is not set.\n", name);
        } else {
            printf("Unknown command.\n");
        }
    }
    return 0;
}
```

OUTPUT:

```
imvarls@LAPTOP-VQ4D70L7: × + v
imvarls@LAPTOP-VQ4D70L7:~$ nano export.c
imvarls@LAPTOP-VQ4D70L7:~$ gcc export.c -o export
imvarls@LAPTOP-VQ4D70L7:~$ ./export
=====
EXPORT COMMAND TESTER
=====

shell> MYVAR=Hello
Unknown command.

shell> test

Child Process:
MYVAR is not set.
Child process completed.

shell> |
```